Search-Engine
Project Documentation

Alejandro Dorich
Last Updated: April 2026
Original Version: January 2025 (University Project)

Table of Contents

1. Introduction..... 1
2. Data Structures..... 1
  2.1. UrlData..... 2
  2.2. Website..... 2
  2.3. UrlTokenScore..... 2
  2.4. UrlCosineScore..... 2
  2.5. Basic Forward Index..... 3
  2.6. Reverse Index..... 4
3. Core Components..... 5
  3.1. Queue..... 6
  3.2. PageRankCalculator..... 6
  3.3. Crawler..... 6
  3.4. StopWords..... 7
  3.5. Index..... 7
  3.6. ReverseIndex..... 8
  3.7. VectorizedForwardIndex..... 9
  3.8. App..... 10
  3.9. DirectedGraph..... 11
4. Class Relationships and Interactions..... 12
5. Restrictions..... 13
6. Project Tests & Analysis of Search Results..... 14
  6.1. Test Classes..... 14
  6.2. Search methods comparison..... 15

Table of Figures

Figure 1: basicForwardIndexMap..... 3
Figure 2: reverseIndexMap..... 4
Figure 3: vectorizedForwardIndex..... 5
Figure 4: net-graph Simplified Version..... 11
Figure 5: Class Relationships and Interactions..... 12
Figure 6: Problem in the Visual Representation of the PageRank Values..... 13

Index of Tables

Table 1: lima-labs-investors | Independently Computed PageRank Values..... 15
Table 2: Expected Results..... 15
Table 3: URL Results for the Reverse Index..... 15
Table 4: URL Results for the Vectorized Forward Index..... 16

1. Introduction

This project was developed in the context of a university project in the field of databases and information systems. The main goal of this project is to create a search engine that enables the crawling, storing and indexing of the titles, paragraphs and text (from here on referred to as the data) found in a website. Furthermore, the search engine should also allow users to perform searches for keywords or phrases within the indexed data. In order to retrieve the results, values such as the TF-IDF (Term Frequency –Inverted Document Frequency), PageRank and cosine similarity have to be calculated. Additionally, the project also has secondary goals, such as depicting and storing a graph displaying all search results, by naming the nodes after the found URLs and connecting these by directed edges corresponding to the references among these URLs. The original project relied on a university-provided intranet (JSON seed files simulating web pages). For this public version, the intranet has been replaced with a minimal, self-contained JSON dataset, reduced in scope but fully functional. Each JSON file in this dataset represents a network, and the collective networks simulate the intranet of the fictional company Lima-Labs. This documentation describes all the components of the project, such as the employed data structures and classes, and the tasks each of these components solves in order to give an overview and provide a clear guideline of all methods implemented in the project.

2. Data Structures

This project implements a variety of different data structures in order to fulfill its goals in an efficient manner. As stated in the introduction, crawling the intranet sites, storing & indexing the data found in these sites, as welll as performing searches within these indexes define the main goal of the project. The following data structures enable the correct implementation of these functions:

2.1. UrlData

The UrlData class serves as the superclass for various URL related classes. It is declared as an abstract class that acts as a container for a single URL. Its subclasses are utilized by different indexes and the Crawler class in order to store all the data

associated with a referenced URL, including the metrics required for indexing and searching through this data.

2.2. Website

The Website class builds upon its superclass UrlData and enables the storage of the data found in a website that is retrieved during the crawling process. Additionally, this class stores the PageRank value calculated by the PageRankCalculator class after completing the crawling process.

2.3. UrlTokenScore

The UrlTokenScore class extends its superclass UrlData by including the following metrics and variables that are required by a reverse index in order to sort URLs by their relevance regarding a search query:

- Token: Represents the token associated with this specific instance of the UrlTokenScore class. The token is one of the terms derived from lemmatizing, tokenizing and removing all stop-words in the data of the referencing URL. The purpose of storing it as a variable of UrlTokenScore is to allow individual UrlTokenScore objects to store all relevant information regarding their TF-IDF values without requiring access to the entire reverse index.
- Term Frequency: The amount that the token associated with this specific instance of the UrlTokenScore class appears in the lemmatized and tokenized data of the referencing URL, after removing all stop-words.
- TF-IDF: The TF-IDF regarding the referencing URL and the associated token.

2.4. UrlCosineScore

The UrlCosineScore class extends its superclass UrlData by including the following metrics that allow sorting URLs based on their relevance to a search query in a ‘vectorized forward index’ (this term is defined in chapter 2.7.):

- Cosine similarity: Stores the cosine value of the angle between two vectors, representing their similarity (see chapter 3.7. for more details).
- PageRank: Rates the importance of a website in a network (see chapter 3.2. for more details)

- Relevance factor: The weighted product of the normalized cosine similarity and PageRank values, serving as the key metric for sorting URLs in a vectorized forward index.

Furthermore, this data structure also stores the normalized values of the cosine similarity and PageRank, as these are required in order to calculate the relevance factor.

2.5. Basic Forward Index

This index serves as a foundation in order to develop more detailed indexes, such as the reverse index and the vectorized forward index. Its purpose is to link all tokens derived from a site to its corresponding URL. In this project, the basic forward index is represented by the variable *basicForwardIndexMap* in the ReverseIndex class and its data structure is defined as Map<String, List<String>>. By using this data structure, the variable can map each URL to a list of tokens which are derived by tokenizing, lemmatizing, and removing stop-words from the crawled data of the corresponding URL. The composition of *basicForwardIndexMap* is depicted in the following figure:

$$\begin{array}{lllllll} URL_1 & (\mathrm{tok}_1, & \mathrm{tok}_2, & \cdots & \mathrm{tok}_{u-1}, & \mathrm{tok}_u) \\ URL_2 & (\mathrm{tok}_1, & \mathrm{tok}_2, & \cdots & \mathrm{tok}_{v-1}, & \mathrm{tok}_v) \\ \vdots & & & & & \\ URL_n & (\mathrm{tok}_1, & \mathrm{tok}_2, & \cdots & \mathrm{tok}_{w-1}, & \mathrm{tok}_w) \end{array}$$

Figure 1: basicForwardIndexMap

In this figure, URL_i represents the *i*-th URL for all $1 \leq i \leq n$ URLs stored in the index and corresponds to the key in the Map. The associated list on the right side of URL_i represents the List<String> value of the *i*-th URL in the Map. This list contains all tokens extracted out of the data of URL_i after applying tokenization, lemmatization, and stop-word removal on this data. By using a specific URL as a key, this implementation allows for constant-time retrieval of all corresponding tokens.

2.6. Reverse Index

The reverse index is represented by the variable *reverseIndexMap* in the ReverseIndex class and its data structure is defined as Map<String, List<UrlTokenScore>>. This structure maps each token to a list of UrlTokenScore objects, corresponding to URLs that contain the token in their data. In other words, each token (obtained after lemmatizing, tokenizing, and removing stop-words during the crawling process) is used as a key, with the associated value being a list of UrlTokenScore objects for URLs where the token appears. This makes it possible to fetch the TF-IDF value corresponding to a specific token and URL, a value necessary in order to perform searches in the reverse index. The composition of *reverseIndexMap* is depicted in 'Figure 2':

$$
\begin{array}{lllll}
tok_1 & (\mathrm{URL}_1, & \mathrm{URL}_2, & \cdots & \mathrm{URL}_{x-1}, \quad \mathrm{URL}_x) \\
tok_2 & (\mathrm{URL}_1, & \mathrm{URL}_2, & \cdots & \mathrm{URL}_{y-1}, \quad \mathrm{URL}_y) \\
\vdots & & & & \\
tok_m & (\mathrm{URL}_1, & \mathrm{URL}_2, & \cdots & \mathrm{URL}_{z-1}, \quad \mathrm{URL}_z)
\end{array}
$$

Figure 2: reverseIndexMap

In this figure, tok_i represents the *i*-th token for all $1 \leq i \leq m$ tokens and corresponds to the key in the map. The associated list on the right side of tok_i represents the List<UrlTokenScore> value of the *i*-th token in the Map. The list contains all UrlTokenScore objects with a URL whose corresponding site contains tok_i in its data.

2.7. Vectorized Forward Index

In this Java project, the term 'vectorized forward index' is defined to describe an index that builds upon the basic forward index by not only confining the value of each key URL to the TF-IDF of the tokens found in its data, but instead storing the value mapped to each key URL as a vector representing the TF-IDF values for all tokens obtained during the crawling process. The vectorized forward index is represented by the variable *vectorizedForwardIndexMap* in the VectorizedForwardIndex class and its data structure is defined as Map<UrlCosineScore, double[]>.This variable sets UrlCosineScore objects corresponding to all crawled URLs as keys and maps each of these keys to a vector of TF-IDF values. As each of these vectors represents the TF

IDF values of all tokens, all vectors in the map share the same dimension. The corresponding token for each TF-IDF value is determined by its index in the *indexValuesForTokens* variable, which assigns a unique index to every token and is also stored in the VectorizedForwardIndex class. The map is depicted in Figure 3:

$$\begin{array}{lllll} URL_1 & (\text{TF-IDF}_1, & \text{TF-IDF}_2, & \cdots & \text{TF-IDF}_{m-1}, & \text{TF-IDF}_m) \\ URL_2 & (\text{TF-IDF}_1, & \text{TF-IDF}_2, & \cdots & \text{TF-IDF}_{m-1}, & \text{TF-IDF}_m) \\ \vdots & & & & & \\ URL_n & (\text{TF-IDF}_1, & \text{TF-IDF}_2, & \cdots & \text{TF-IDF}_{m-1}, & \text{TF-IDF}_m) \end{array}$$

Figure 3: vectorizedForwardIndex

In this figure, URL_i represents the *i*-th UrlCosineScore object with $1 \leq i \leq n$ and n being the total amount of crawled URLs stored in the index. Each UrlCosineScore object corresponds to a single crawled URL and allows the storage of the cosine similarity, PageRank and the relevance factor, as explained in chapter 2.4. This makes it possible to easily calculate and fetch the relevance factor corresponding to a specific token and URL, a value necessary in order to perform searches in the vectorized forward index. Furthermore, this figure clearly illustrates the major difference between this index and the basic forward index, as each URL is clearly mapped to all tokens, including those not explicitly mentioned in the URL. Because of this, all vectors of TF-IDF values are defined as a double[],since the dimension of each vector is the same. The *j*-th TF-IDF value for URL_i corresponds to the *j*-th token in *indexValuesForTokens*, for all $1 \leq j \leq m$ tokens.

3. Core Components

This project implements a variety of different classes and methods that implement the aforementioned data structures in order to create a search engine for the intranet and store the graph displaying all search results, as stated in the introduction. In order to avoid redundancies, the classes UrlData, Website, UrlCosineScore and UrlTokenScore are not listed in this chapter, as they primarily serve as data structures.

3.1. Queue

This class builds a customized queue that allows the storage and management of String objects. This queue only allows the adding of String objects whose content has never been added to the queue at any given moment. If a String object with the same content has been added to the queue and later removed, trying to re-add it will have no effect, leaving the queue unchanged. In addition to returning the number of elements stored in the queue, this class includes only one more method, the 'poll' method. This method removes and returns the first String object from the queue. In this project the Queue class functions as an essential component of the Crawler class, since it is used to manage all the URLs that have to be crawled. By preventing the re-addition of String objects with the same content, it avoids infinite loops that can occur when multiple sites reference the same external link.

3.2. PageRankCalculator

This class enables the calculation of the PageRank value for each Website object as defined in by the PageRank algorithm. It initializes the PageRank value for each site with 1 divided by the amount of all crawled sites. Afterwards, it iteratively calculates the PageRank value for each site using the following equation:

$$PR_i = d \cdot \sum_{j \in L_i} \frac{PR_j}{c_j} + (1-d) \cdot \frac{1}{N}$$

Where d represents the damping factor set at 85%, PR_i the PageRank of the *i*-th website, PR_j the PageRank of the *j*-th website, L_i the amount of sites that reference the *i*-th site, N the amount of all crawled through sites and c_j the amount of external links found in the *j*-th site. Finally, it adds the difference of the old and newly calculated PageRank value for each site. If this difference is lower than 0.0001, the algorithm finishes. The relevance of the damping factor is briefly explained in chapter 6.1.

3.3. Crawler

This class is responsible for crawling the entire intranet and all the seed-URLs and external links found in the JSON files the intranet encompasses. The storage of all the gathered data during this process in a variable setup as a list of Website objects called *crawledSiteData* establishes the foundation to create diverse types of indexes that allow managing and performing searches within this data. In order to crawl the

intranet, this class employs the Queue class that it uses in order to manage the list of URLs that need to be crawled, thus ensuring the evasion of redundant crawling by not crawling through the same site more than once. Moreover, the Crawler class is also able to return the amount of crawled pages per JSON file, as well as the number of external links found in these crawled pages. This class implements the PageRankCalculator and stores the PageRank value as a variable for each site in *crawledSiteData*. Implementing the PageRank calculation in this class is logical, as it allows the PageRank to be directly added to all Website objects in *crawledSiteData*. This approach makes the data accessible to various other classes and decreases the steps required to calculate these values. For instance, the vectorized forward index uses the *crawledSiteData* list in order to retrieve the PageRank values and incorporate these values into the index.

3.4. StopWords

The StopWords class is a utility class that contains all stop-words to be eliminated after the tokenization and lemmatization processes, performed to create various types of indexes from the data retrieved during the crawling process. It is essential for performing searches in these indexes, as the search query must also undergo tokenization, lemmatization, and stop-word removal. The actual stop-words are stored in the variable *stopWordsSet*, which is declared as a Set<String>. The Set data structure makes sense in this case, since the sequence in that the words are listed is irrelevant. The StopWords class also provides the 'deleteStopWords' method, that deletes all stop-words contained in *stopWordsSet* out of the given String argument. This method is useful to the Index class and all of its subclasses.

3.5. Index

The Index class is an abstract base class designed to support search operations on the data gathered by the crawler. It provides common methods to its subclasses, such as proving if a given String array is only composed out of tokens and transforming a given String element into a list of tokens.

3.6. ReverseIndex

The ReverseIndex class is in charge of creating both a basic forward index and a reverse index. This class utilizes the data retrieved during the crawling process in

order to create the basic forward index, which is stored in the variable *basicForwardIndexMap* as mentioned in chapter 2.5 and illustrated in figure 1. This index is then in turn used to create the reverse index, which is stored in the variable *reverseIndexMap* as mentioned in chapter 2.6 and illustrated in figure 2. The reason for leveraging one index to construct the other one stems from the project's requirements, which specifically mandate the implementation of the basic forward index. Additionally, given the general structure of the basic forward index, it allows the index to be easily converted into a reverse index. After having created the reverse index, the class allows searching for a specific query in the index. This is achieved by tokenizing and lemmatizing the query, as well as removing all the stop-words that these tokens contain. The resulting tokens are searched in the *reverseIndexMap*, and all UrlTokenScore objects mapped to these tokens are retrieved and added to a result list. If a retrieved UrlTokenScore object references a URL already present in the list, its TF-IDF value is added to the TF-IDF value of the existing object instead of adding a duplicate entry. This approach ensures that no URL appears more than once in the search results while increasing the TF-IDF score for URLs containing multiple relevant tokens. Finally, the list is sorted by the TF-IDF values of the UrlTokenScore objects and returned as the search results. Setting List<UrlTokenScore> as the return type is useful, since the sequence of the objects is relevant and the amount of resulting objects is not known before conducting the search. Moreover, the UrlTokenScore objects make it possible to prove if the results are correct since they contain the TF-IDF values for each URL.

3.7. VectorizedForwardIndex

The VectorizedForwardIndex class is in charge of creating a vectorized forward index as discussed in chapter 2.7. In order to create this index, this class requires an instance of ReverseIndex in which the reverse index has already been created and stored in the variable *reverseIndexMap*, as discussed in chapter 3.6. Once created, the vectorized forward index is then stored in the variable *vectorizedForwardIndexMap* (refer back to Figure 3 for a detailed visualization of the variable's structure). The reason this class requires an instance of ReverseIndex in order to create the vectorized forward

index lies in the project's requirements. The requirements state that both a reverse index and a vectorized forward index have to be created within the App class (see chapter 3.8.), whose final goal is to search the intranet for a specific search query and return the search results. Using the already created ReverseIndex instance in order to create the vectorized forward index takes advantage of the existing reverse index and avoids redundancies such as having to calculate the TF-IDF values again. In order to make it possible to perform searches within the vectorized forward index, this class is also responsible for the calculation of the cosine similarity and the relevance factor. The latter being calculated as the weighted product of the normalized values of the cosine similarity and PageRank. Here the weight is set as 50% for both the normalized cosine similarity and PageRank values, as this approach ensures a balanced contribution of both metrics. Using the calculated relevance factor values, the class can perform searches for a specific query in the index and return a list of UrlCosineScore objects sorted by these values by using the 'searchQuery' method. As in the ReverseIndex class, each search is achieved by tokenizing and lemmatizing the query, as well as removing all the stop-words that these tokens contain. The resulting tokens are used as a basis in order to create a vector representation of the query. This vector is then in turn used in order to calculate the cosine similarity for each UrlCosineScore in the variable *vectorizedForwardIndexMap*. The UrlCosineScore objects in the index with a cosine similarity unequal to 0 are added to a result list. Finally, the list is sorted by the relevance factor values of the UrlCosineScore objects and returned as the search results. Similarly to the search results in the ReverseIndex class, setting List<UrlCosineScore> as the return type is useful, since the sequence of the objects is relevant and the amount of resulting objects is not known before conducting the search. Additionally, the UrlCosineScore objects make it possible to prove if the results are correct since they contain the PageRank, cosine similarity (including the normalized values), as well as the relevance factor. On top of constructing a vectorized forward index and performing searches for a specific query in this index, this class also addresses other extra tasks of this project. In addition to the search method presented above, this class also enables a search for a weighted query, that is implemented in the 'searchWeightedQuery' method. This method enables searching with a weighted query by defining the query's data structure as

Map<String, Double>. By setting the query as a map, each token can be directly linked to its given weight, meaning that the significance for each token can be provided by the user.

Concerning the efficiency of the search algorithm, this class also provides the method ‘normalizeVectors’ that simplifies the calculation of the cosine similarity. This simplification was implemented as a method, as the university project required both, the inefficient and efficient calculations of the cosine similarity. It is achieved, by normalizing all the vectors within the index and the vector corresponding to the weighted query, so that their Euclidean norm equals 1. For clarification, the cosine similarity is calculated as follows:

$$\frac{\sum_{i=1}^{n} a_i \cdot b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \cdot \sqrt{\sum_{i=1}^{n} b_i^2}}$$

Whereas $\sqrt{\sum_{i=1}^{n} a_i^2}$ and $\sqrt{\sum_{i-1}^{n} b_i^2}$ represent the Euclidean norm of vector a and vector b, respectively. Hence, if the Euclidean norm of all vectors is set to 1, the cosine similarity is reduced to the sum of the product of the corresponding components of the vectors. This class achieves this normalization, by dividing all entries in each vector through the Euclidean norm of the entire vector. By doing so, it assures the Euclidean norm of all normalized vectors to be 1.

3.8. App

The App class serves as the main class for the project. It crawls the entire intranet and stores all gathered data in a vectorized forward index. As described in previous chapters, this process involves first creating a basic forward index, then using this index in order to generate a reverse index, and finally constructing the vectorized forward index using the information stored in the reverse index. Additionally, the class allows a user to input a search request and returns all relevant URLs along with the titles of their corresponding sites, according to the vectorized forward index (see chapter 3.7 for an overview of the search process). Users can choose between normal or weighted search requests and submit as many search request as needed.

3.9. DirectedGraph

As stated in the introduction, some secondary goals of this project include depicting and storing a graph that displays all search results and diverse metrics. This class solves these tasks by declaring a variety of different methods and implementing them in its main class. In this process, a total of four graphs are stored in a folder named 'figures' within the project's directory. The graphs are created incrementally, with each subsequent graph building upon the previous one. First of all, the first 8 sites in the Lima-Labs customer network are crawled. All external links found on the crawled sites that are not part of the original 8 crawled sites are removed to ensure the correct calculation of the PageRank values. The first graph is created out of this data, with each URL represented as a node. The edges in the graph illustrate the references between the sites corresponding to the URLs. The graph is stored under the name ‘net-graph’. The following figure illustrates a simplified example of the graph, under the assumption that only two sites were crawled, each containing an external link to the other:

Figure 4: net-graph
Simplified Version

In order to build the second graph, a reverse index is created out of the data obtained during the crawling process. The term ‘support’ is searched for in this index and all relevant TF-IDF values for the 8 crawled sites are added to the nodes of the ‘net-graph’. The TF-IDF values are displayed below each node’s URL. Additionally, the search term 'support' is shown in the top-left corner of the graph. This new graph is stored under the name of the search term, ‘support’. As for the next graph, the main class adds all PageRank values to the edges of the graph and scales the nodes according to the PageRank value of their corresponding site, as calculated directly after the crawling process. This graph is stored as ‘support_page-rank’. In order to build the fourth and final graph, a vectorized forward index is created out of reverse index. The term ‘support’ is then also searched for in this index and all relevant

relevance factor values for the 8 crawled sites are added to the nodes of the 'net-graph'. The relevance factor values are displayed below each node's TF-IDF value and the 3 most relevant URLs according to these values are highlighted by changing the color of their nodes to light green. Finally, the graph is stored as 'support_top3'.

4. Class Relationships and Interactions

To better illustrate the relationships and dependencies among the classes and data structures used in this project, as described in Chapters 2 and 3, refer to Figure 5:

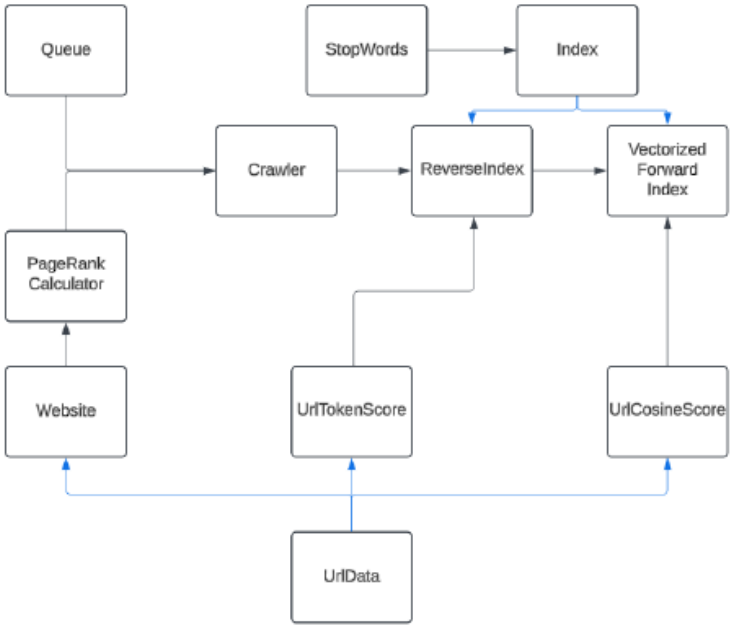

Figure 5: Class Relationships and Interactions

While the black edges represent dependencies, the blue ones illustrate the inheritance among the classes implemented in this project. This being said, not all the classes are illustrated in this figure. The classes App (see chapter 3.8) and DirectedGraph (see chapter 3.9) are excluded, as their inclusion would make the figure overly complex and difficult to interpret. The App class serves as the primary class for the project, utilizing all other classes to achieve the main objectives. Similarly, the DirectedGraph class plays a central role in fulfilling nearly all secondary objectives and therefore also relies on the implementation of all other classes.

5. Restrictions

While all the primary and secondary goals set by the university for this project were successfully achieved, there is a limitation regarding the PageRank representation in the graph's edges. While this project succeeds in displaying the PageRank values on the edges of the graph, reflecting the fraction of the PageRank passed from one node to another, some PageRank values are not depicted properly. This problem is shown in figure 6:

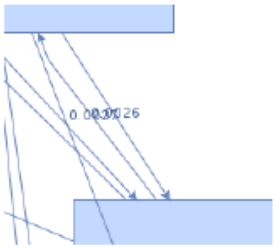

Figure 6: Problem in the Visual Representation of the PageRank Values

For a few edges, the PageRank values are overlaid on top of the values of other edges, making it difficult to clearly read the actual values. The root cause of this problem lies in the project's requirements, which specify that the graph must be constructed as an mxGraph using the JGraphX library. However, no suitable layout or alternative solution could be identified in the documentation or user manuals for both mxGraph or JGraphX to address this issue effectively. On the positive side, not many edges are affected by this issue; the vast majority of PageRank values is illustrated correctly and can be interpreted without any problems.

6. Project Tests & Analysis of Search Results

This project includes a variety of tests to verify that all classes and methods function as intended. Additionally, selected search results are analyzed to illustrate the differences between the search methods of the reverse index and the vectorized forward index.

6.1. Test Classes

This project includes the following tests in order to check if all classes function as expected:

- CrawlerTests: Tests the correct functionality of the crawler by analyzing if the number of crawled sites and the number of links found during the crawling process is correct.
- ReverseIndexTests: Tests if the reverse index works as intended by checking if all tokens are mapped to the correct URLs and if the TF-IDF values are correct.
- PageRankTests: Verifies whether the PageRankCalculator class functions as intended by comparing the calculated PageRank values with the expected (independently computed) values for the sites in the Lima-Labs investors network (see Table1). Moreover, this test emphasizes the need to include a damping factor (see chapter 3.2.) in the PageRank calculation. Without the damping factor, the difference between the old and newly calculated PageRank values never becomes less than 0.0001, preventing the algorithm from converging and causing it to run indefinitely.
- CosineTests: Verifies whether the 'calculateCosineSimilarity' method of the VectorizedForwardIndex class works as intended by comparing the calculated cosine similarity values with the expected values for a selected set of vectors.
- SearchTests: Verifies whether the search method of both the ReverseIndex and the VectorizedForwardIndex classes delivers the expected URLs for a given query in each network of the Lima-Labs intranet.

All presented classes successfully pass all provided tests, demonstrating their correct functionality and reliability. This indicates that the core objectives of the project have been properly implemented and fulfilled.

	Iteration											
	0	1	2	3	4	5	6	7	8	9	10	11
index	0.20000	0.28500	0.25289	0.26312	0.25926	0.26095	0.26022	0.26052	0.26040	0.26045	0.26043	0.26044
funding	0.20000	0.22833	0.22833	0.22720	0.22526	0.22704	0.22614	0.22652	0.22636	0.22643	0.22640	0.22641
board	0.20000	0.14333	0.15136	0.14454	0.14873	0.14699	0.14763	0.14738	0.14748	0.14744	0.14746	0.14745
financials	0.20000	0.20000	0.21606	0.20923	0.21310	0.21082	0.21195	0.21145	0.21166	0.21158	0.21161	0.21160
pitch	0.20000	0.14333	0.15136	0.15591	0.15365	0.15420	0.15406	0.15413	0.15409	0.15411	0.15410	0.15410
Delta PR	-	0.22667	0.06422	0.02957	0.01611	0.00803	0.00354	0.00150	0.00063	0.00026	0.00011	0.00005
DeltaPR < 0.0001?												finished

Table 1: lima-labs-investors | Independently Computed PageRank Values

6.2. Search methods comparison

As elaborated in chapters 3.6 and 3.7, the search functionality of the VectorizedForwardIndex class differs from the one of the ReverseIndex class, by taking the PageRank and cosine similarity values into account when depicting search results. In order to fully illustrate the differences between the search methods used by both classes, a brief comparison is depicted for the Lima-Labs internal network. Using the data collected during the crawling process for this network, both the search methods in the ReverseIndex and VectorizedForwardIndex classes successfully return the expected URLs when searching for the defined Query-Tokens declared in the lima-labs-internal JSON file. The primary difference lies in the order in which these URLs are listed. Tables 1 to 3 illustrate expected results and the ranking of the actual resulting URLs (from top to bottom), along with the key metrics used to determine this order:

URL Nr.	Title
1	http://localhost:8080/pages/internal/security.html
2	http://localhost:8080/pages/internal/careers.html
3	http://localhost:8080/pages/internal/products.html
4	http://localhost:8080/pages/internal/blog.html

Table 2: Expected Results

URL Nr.	Title	TF-IDF
1	http://localhost:8080/pages/internal/security.html	0.1222
3	http://localhost:8080/pages/internal/products.html	0.0458
4	http://localhost:8080/pages/internal/blog.html	0.0416
2	http://localhost:8080/pages/internal/careers.html	0.0367

Table 3: URL Results for the Reverse Index

URL Nr.	Title	Relevance Factor	Cosine Similarity	PageRank
1	http://localhost:8080/pages/internal/security.html	0.8767	0.3468	0.1040
3	http://localhost:8080/pages/internal/products.html	0.6572	0.1091	0.1381
2	http://localhost:8080/pages/internal/careers.html	0.5305	0.1023	0.1057
4	http://localhost:8080/pages/internal/blog.html	0.3677	0.1073	0.0588

Table 4: URL Results for the Vectorized Forward Index

While both indexes return the same top two URLs, their ranking diverges for the remaining results. Specifically:

- The ReverseIndex ranks URL no. 2 as the least relevant due to its lowest TF-IDF score, indicating minimal textual relevance to the query.
- The VectorizedForwardIndex, however, places URL no. 4 in the last position. This discrepancy arises from the different relevance metrics applied:
  - In the ReverseIndex, ranking is solely determined by TF-IDF: URL no. 4 outperforms URL no. 2, placing it higher.
  - In the VectorizedForwardIndex, the combined relevance factor (Cosine Similarity & PageRank) dominates:
    - URL no. 2 benefits from a higher PageRank, driven by three incoming references, which compensates for its slightly lower cosine similarity.
    - URL no. 4 is referenced by only one other site, resulting in a low PageRank, despite its slightly better cosine similarity, resulting in the lowest overall relevance factor.

Thus, the VectorizedForwardIndex also takes link-based authority (PageRank) into account, while the ReverseIndex purely focuses on textual relevance (TF-IDF)."